Auditable Public-Procurement Question Answering with Typed Planning and Deterministic KG Execution

Hai Zhou *uceeh01@ucl.ac.uk*
University College London

Abstract

Answering public-procurement questions often requires complete sets, role-sensitive joins, and exact aggregation. Top-k retrieval can miss records that change the answer, while direct language model generation provides no reliable account of how a value was obtained. We present a hybrid LLM–knowledge-graph system over 215,221 UK contract-award records. A structured understander and conditional typed planner propose a computation; deterministic code grounds it to a provenance-preserving property graph, executes it over the complete matching population, and either releases an answer or abstains. The same frozen backend and typed answer contracts support benchmark construction and offline filtering of training plans. On 2,025 procurement questions not used in the 260-question model-selection subset, the best fully local Qwen3-8B pipeline is correct on 85.73% under exact answer-or-correct-non-release scoring, compared with 69.19% for its cloud teacher and 38.42% for a stronger retrieval-augmented baseline. A single-call algebra planner reaches 78.31% on a 922-question compositional diagnostic and 77.11% under independently gated paraphrases. Porting the same typed algebra to WikiTableQuestions raises official test accuracy from 22.51% for the base model to 51.80% with gold-program supervision. These results show that learned planning and deterministic execution complement each other. They also identify the remaining boundary: executable, answer-matching plans can still omit a question constraint, and contract-level provenance is not yet a complete source-document citation system.

1 Introduction

Public procurement data describe buyers, suppliers, awards, values, dates, categories, and source notices. The records are public, but many useful questions are not record lookups. An auditor may ask which buyers share suppliers with a named authority, whether all awards in a period satisfy a condition, or which category has the largest complete total. Such questions combine entity roles, multi-hop relations, set operations, and aggregation.

Retrieval-augmented generation (RAG) is a poor execution model for these questions. Let $R(q)$ be the records that satisfy question q and let $\widehat{R}_k(q)$ be the top-k retrieved records. Even when every retrieved row is relevant, $\widehat{R}_k(q) \subset R(q)$ does not imply $|\widehat{R}_k(q)| = |R(q)|$, nor does it preserve a sum, ranking, or set difference. Increasing k reduces but does not remove this problem. A language model can instead generate a database or graph query, but a syntactically valid query may still reverse buyer and supplier roles, select the wrong return field, or omit a year. Executability is therefore necessary but not sufficient.

This work builds an end-to-end alternative for UK public procurement. Open Contracting Data Standard releases are reduced to a versioned snapshot, organisations are resolved conservatively, and contract, organisation, category, and evidence objects are stored in a Parquet-backed property graph. A language model translates a question into a typed computation. Deterministic code then grounds fields and values, retrieves the complete matching population, executes joins and aggregates, and applies release checks. The model proposes a computation; it is never the source of the factual value returned to the user.

The implemented runtime has two learned stages. A structured understander first predicts an answer signature and a typed intent program. Simple valid intents are compiled directly. More complex or vetoed intents are

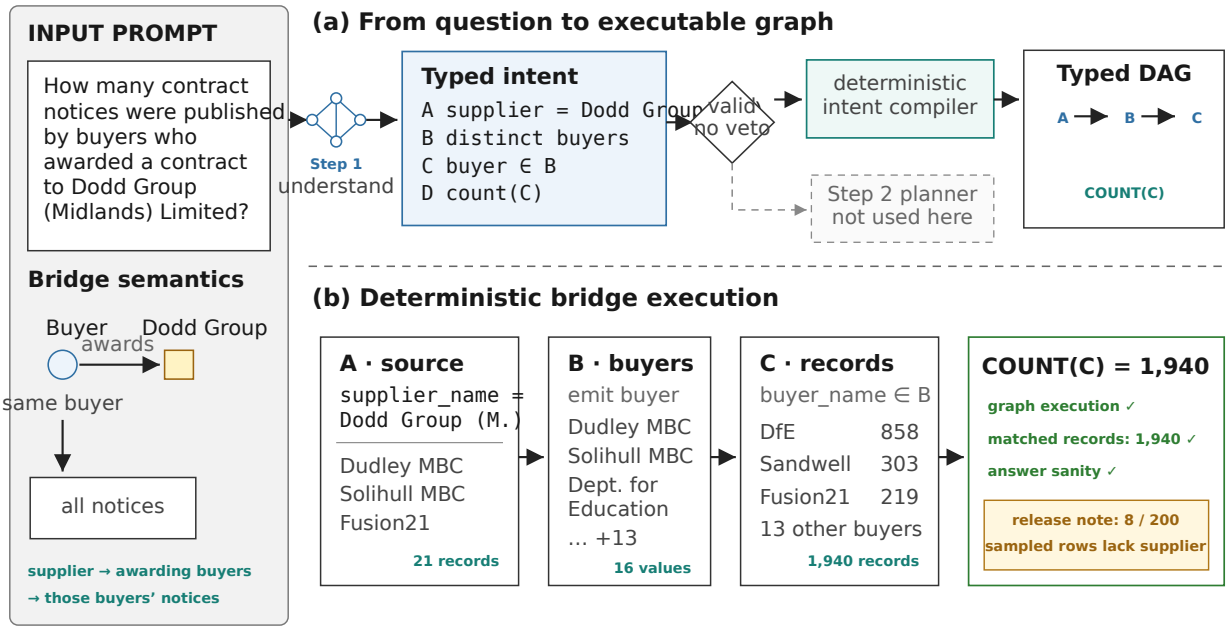

Figure 1: Control path of the full procurement runtime. The understander is always called; a valid compilable intent takes the deterministic fast path, while other intents are passed to the typed graph planner. Both branches are grounded and executed on the frozen graph before release checks. Only a structured, repairable failure may trigger the bounded replan.

passed to a graph planner that makes intermediate sets and their dependencies explicit. Both routes enter the same grounding, execution, evidence, and abstention path. The resulting trace records the grounded constraints, intermediate population sizes, and a bounded sample of contract-level provenance. A failed structural or execution check can trigger a limited replan; unsupported, ambiguous, and clean no-result questions terminate without searching for a more convenient answer.

The project also studies a simpler, single-call planner. It maps a question directly to a recursive typed algebra and is evaluated without the understander, graph planner, or repair loop. This protocol isolates compositional program generation and makes the same algebra portable to tabular data. It is used for the 922-question PACS diagnostic and for WikiTableQuestions (WTQ), whereas the main procurement experiment evaluates the complete two-stage runtime. Keeping these protocols separate is important: they share typed execution, but they do not measure the same system.

The evaluation addresses three questions:

- **RQ1: End-to-end QA.** Does the hybrid system improve exact answers and correct abstention over closed-book, RAG, plan-guided RAG, cloud, and deterministic KG-only baselines?
- **RQ2: Planning and composition.** Which training stages improve the full runtime, and does a specialised algebra planner retain its gains on new compositions and paraphrases?
- **RQ3: Reliability and portability.** Does the runtime withhold answers for declared non-answer states, what provenance does it preserve, and which planning components transfer to a different structured domain?

The evidence is deliberately layered. The main procurement result excludes the 260 questions used for checkpoint selection and contains 2,025 remaining questions. The 260-question subset is retained only for paired checkpoint and baseline analysis. PACS tests composition in 922 procurement questions, and WTQ

tests the algebra on 4,344 unseen-table questions with the official evaluator. Targeted audits then test deterministic grounding and training selection without presenting either as a separate end-to-end benchmark.

The paper makes three contributions. First, it provides a reproducible data and representation pipeline from raw procurement releases to a provenance-preserving property graph. Second, it implements a hybrid QA runtime that combines typed understanding and planning with exhaustive deterministic execution, evidence traces, bounded repair, and explicit non-answer states. Third, it evaluates the system at scale and separates end-to-end performance, checkpoint effects, compositional planning, cross-domain transfer, and grounding diagnostics instead of treating them as one accuracy number.

2 Related Work

Executable question answering. Semantic parsers map language to programs over tables, databases, or knowledge graphs (Pasupat & Liang, 2015; Wang et al., 2020; Cheng et al., 2023). Table-specialised models instead encode the table jointly with the question (Herzig et al., 2020; Yin et al., 2020; Liu et al., 2022), while recent LLM systems generate SQL, logical forms, relation paths, or structured-data calls (Jiang et al., 2023; Luo et al., 2024a;b). These methods establish that symbolic execution can move factual calculation outside the language model. Our problem adds a data-layer requirement: entity aliases, buyer/supplier roles, release versions, and amount provenance must be defined before a program has a stable meaning.

Schema linking and constrained execution. Schema-aware encoders and linkers reduce field and relation errors, and constrained decoders prevent illegal output prefixes (Wang et al., 2020; Scholak et al., 2021). Execution can reject candidates that fail in the target environment. Learning from Executions uses this signal for semi-supervised parsing, while LEVER learns to rank programs from their code and results (Wang et al., 2021; Ni et al., 2023). These controls address different failure levels. Grammar checks establish well-formedness; schema grounding establishes that references exist; execution establishes a denotation on one database. Our runtime combines these levels and adds explicit release and abstention states, but does not treat executability as semantic proof.

Composition and weak supervision. Multi-step table and graph QA benefits from explicit intermediate operations or relation paths (Wang et al., 2024; Luo et al., 2024b). When only an answer is observed, however, several programs can share that answer. Fictitious worlds, execution signatures, and structural filters have been used to remove such spurious programs (Pasupat & Liang, 2016; Lee et al., 2023). This issue is relevant to our offline curation because an oracle-matched plan may omit a constraint that happens to be redundant on the frozen graph. We therefore separate type validity, runtime verification, answer match, and constraint retention.

Answerability and provenance. Unanswerable and selective QA evaluates whether a system withholds unsupported answers rather than optimising accuracy only on answerable items (Rajpurkar et al., 2018; Kamath et al., 2020). In knowledge-graph QA, provenance can be expressed as relation paths; in document RAG it is usually a retrieved passage. Our system stores contract-level provenance and execution traces and evaluates explicit ambiguous, unsupported, and no-result classes. It does not yet evaluate the correctness of citations into source-document text, so we use *provenance-linked* rather than *citation-verified*.

Procurement knowledge graphs. TheyBuyForYou shows how public-contract and company data can be integrated into an open linked-data platform (Soylu et al., 2022). We focus on natural-language analysis over a fixed UK procurement snapshot. The research gap is the combination of a reproducible procurement data layer, typed multi-hop planning, exhaustive aggregation, explicit provenance, and selective answering in one evaluated system. Existing work supplies strong components, but does not by itself solve the completeness and auditability requirements of this setting.

3 Task, Data, and Evaluation Contract

Table 1: Frozen data and benchmark artifacts. Coverage counts refer to contract nodes with at least one corresponding edge.

Graph object	Count	Coverage	Benchmark split	Count	Status
Contract nodes	215,221	–	Train	9,267	mixed
Organisation nodes	131,502	–	Dev-select	671	mixed
CPV nodes	3,870	–	Dev-tune	556	mixed
Evidence nodes	535,731	–	Dev-smoke	49	mixed
Contracts with buyer	215,218	100.00%	Frozen final partition	2,285	1,925/360
Contracts with supplier	204,186	94.87%	Main test after dev removal	2,025	1,725/300
Contracts with evidence	215,202	99.99%	Total	12,828	–

3.1 Task definition

Let K be a frozen procurement snapshot and q a natural-language question. An answerable item has a typed oracle y and an executable specification $z^\star$. A system predicts either an answer $\hat{y}$ or no answer. Exact correctness is defined by answer type: Booleans must be Booleans, numbers use decimal equality, and set/list answers compare normalised members without order unless ranking is requested. A non-answerable item is correct only when the runtime releases no answer.

The procurement benchmark distinguishes three non-answer states. *Unsupported* questions request an operation outside the runtime contract. *Ambiguous* questions have more than one valid interpretation or factoid. *No-result* questions are well formed but have an empty denotation in K. These labels test failure detection; they do not assert that the real world has no answer.

3.2 Graph and benchmark snapshot

The source is a set of yearly Open Contracting Data Standard releases for 2022–2026 (Open Contracting Partnership, 2024). The build retains the latest dated release for each OCID and records all source years. It produces a Parquet-backed property graph rather than an RDF or SPARQL service. Contract nodes are keyed by OCID and award identifier. Organisation, CPV, and evidence nodes are linked through buyer, supplier, category, and provenance edges. Table 1 gives the sizes used by the experiments.

The query view flattens this graph to one record per contract-award while retaining stable node and provenance identifiers. Counts deduplicate contract identifiers. Value projections return distinct stored strings, which may still contain unresolved aliases and are not guaranteed to be distinct legal organisations. Amounts record their source and currency. Award- and contract-derived values may be marked additive; tender-level fallback values are not. No currency conversion is performed, so cross-currency totals are excluded from the paper's main claims.

3.3 Guarantee levels

We use five terms consistently. A program is *valid* when its operators and types compose. It is *grounded* when fields, values, roles, guards, and dependencies map to the backend contract. It is *executable* when deterministic evaluation reaches an allowed state. *Runtime verified* means preflight, postflight, answer-sanity, and evidence-release checks pass without access to y. *Oracle correct* means that the executed result matches y during offline curation or evaluation. None of these conditions alone guarantees that every question constraint appears in the plan or that the frozen source is complete.

4 Method

The project contains an offline construction path and an online answering path. Offline code builds the graph, creates executable benchmark specifications, and filters model-generated plans for training. Online, learned components propose a typed computation and deterministic components decide what it means on the frozen

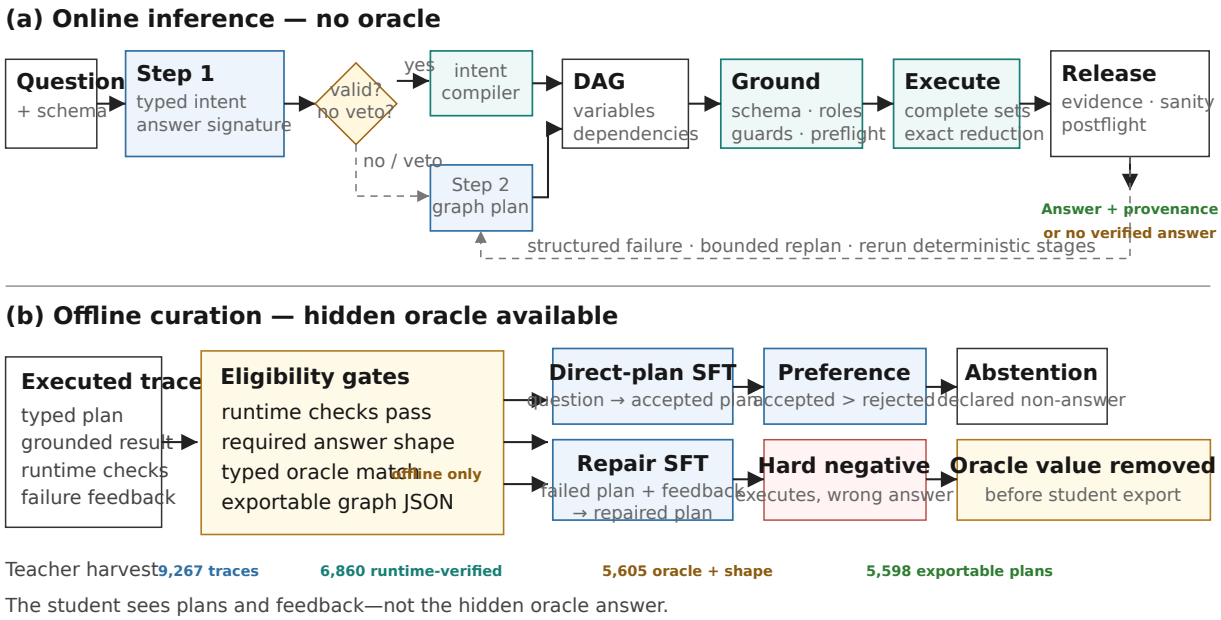

Figure 2: Online inference and offline supervision construction. The online runtime compiles, grounds, executes, and checks a proposed computation without access to an oracle. Offline curation additionally compares the typed result with a hidden oracle before exporting direct plans, repairs, preferences, hard negatives, or abstentions. A repair reruns every deterministic stage.

graph and whether its result may be released. Gold answers are available only offline. Figure 2 separates these two authority paths.

4.1 Procurement data pipeline

Ingestion and snapshot selection. The loader streams compressed JSON Lines files and flattens OCDS releases, parties, awards, and contracts. Blank and malformed lines are logged and skipped. A contracting process can be published more than once. For each OCID we retain the release with the latest parseable date and store the set of source years in which that OCID appeared. Extraction later accepts only this selected (OCID, release-id) pair, which prevents an older version from re-entering the graph.

Entity resolution. Organisation resolution favours precision. A recognised official scheme and identifier forms a canonical node. A platform-specific FTS identifier is linked to an official node only when all observations with the same OCID, normalised name, and party role identify one candidate. Unresolved observations next pass through a government lookup, exact normalised name plus region, and finally exact name when no region is available. Residual clusters receive stable hashed identifiers, and empty normalised names remain singletons. Jaro–Winkler similarity of at least 0.92 produces a review candidate but never an automatic merge. A later union–find pass applies reviewed edges only after checking cluster-level identifier conflicts. Every raw identifier is retained in an alias map.

Extraction, graph construction, and validation. The extractor creates tender, lot, award, bid-statistic, text-evidence, and document-metadata tables. Amount selection follows award, linked contract, then tender precedence and preserves value, currency, and source. Organisation, contract, CPV, and evidence nodes are connected by BUYER-OF, SUPPLIER-OF, CATEGORISED-BY, and EVIDENCE-FOR edges. Exact lot evidence is preferred; an OCID-level link is used when no lot link is available. Before writing the graph, the build checks primary keys, edge endpoints, alias targets, JSON fields, additivity policy, and award coverage. Buyer,

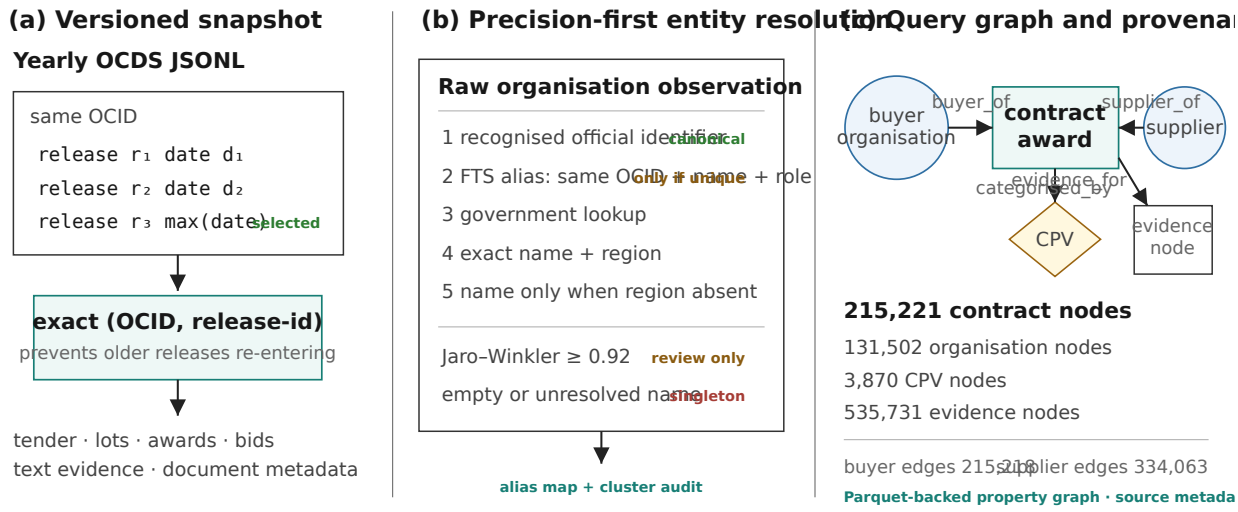

Figure 3: Procurement data and representation pipeline. Version selection precedes exact rich extraction; precision-first entity resolution preserves an alias audit; and the resulting Parquet-backed property graph retains category and contract-level provenance links. Fuzzy similarity creates review candidates rather than automatic merges.

supplier, category, and evidence coverage are reported separately rather than silently filling missing values. Figure 3 summarises the representation path.

4.2 Executable benchmark construction

Questions begin as deterministic *answer specifications*, not as free-form model outputs. A specification contains literal constraints, an answer operation and field, a deduplication key, expected status, and provenance. Stage 1 executes each candidate on the production record backend and a separate reference index. It rejects conflicting constraints, inconsistent record IDs, non-unique factoids, unstable answers, and unsafe sums. Stage 2 renders an accepted specification as a natural question. A second gate checks identifier leakage, literal preservation, answer type, operation, and visible constraints. The naturaliser never writes the oracle.

The initial process retained 9,044 of 10,000 specifications after naturalisation. Targeted builders then added coverage-fixed, extended-operation, bridge, identity, and explicit non-answer examples. The final 12,828 rows are assigned by whole plan group, so a plan identifier cannot occur in both training and evaluation. Curation also balances operation and difficulty buckets and diversifies question surfaces. The current artifact has no train–final plan-ID overlap, although two exact question–answer strings and five deduplication groups cross that boundary; we report this in Section 7.

4.3 Full procurement runtime

Structured understanding. The first model receives the question and a keyword-selected schema slice, but no database rows or oracle. It returns a JSON intent program containing an answer signature, typed steps, literal constraints, role-bearing entity mentions, dependencies, and any unsupported or ambiguous reason. The validator checks step references, return type, answer signature, and dependency order.

Direct compilation and conditional graph planning. A valid intent is compiled deterministically unless it triggers a fast-path veto. The two current vetoes prevent an unsupported cue from coexisting with an answering plan and prevent a request for record count from compiling to an entity-set count. If direct compilation fails or is vetoed, a second model receives the question, the structured briefing, schema context, and the supported capabilities. It returns a typed variable graph.

Each graph variable has a kind, semantic role, constraints, and dependencies. A relation binds a field emitted by one variable into a downstream filter. The return specification names the input variable, operation, answer field, comparator or grouping key, and guards. For example, a buyer–supplier–buyer question is represented as

$$R_b \longrightarrow S_b = \text{distinctSupplier}(R_b) \longrightarrow R[\text{supplier} \in S_b] \longrightarrow \text{distinctBuyer}(R) \setminus \{b\}.$$

Before compilation, a consistency checker compares the graph with explicit years, CPV codes, categories, titles, numeric literals, role cues, comparison direction, and the requested count target. In evaluation, the planner may draw two structural samples, but resampling occurs only after a compile or consistency failure that can be detected without an oracle.

Hard grounding and preflight. Grounding maps aliases to allowed backend fields, checks value shapes, resolves buyer and supplier mentions, verifies emitted and bound fields, forces exhaustive reduction, and inserts the additive guard required by monetary sums. Field grounding uses declared aliases and local character n-gram candidates with type checks; entity resolution requires a unique acceptable match rather than blindly selecting the first candidate. A cycle, missing variable, incompatible dependency, ambiguous entity, illegal field, or unsafe aggregate produces a structured failure.

Deterministic execution. Variables execute in dependency order over the complete contract-record universe. Source variables filter records, entity variables emit distinct values, dependent record variables consume those values through set membership, and scalar variables count, sum, compare, rank, or select. Counts deduplicate contract identifiers, sums use decimal arithmetic, and ties use stable ordering. Independent variables at the same dependency level may run concurrently without changing their denotation. Each variable trace records its grounded constraints, status, dependencies, output size, and emitted field.

Evidence, release, and bounded repair. Preflight checks schema legality, conflicts, and exhaustive execution. Postflight checks result cardinality and population disclosure; answer-sanity and evidence rules then decide whether an answer card may be returned. The card contains the answer, exact population statistics, and a bounded sample of matching contract identifiers. The sample supports audit and replay but is not the complete contributing population for a large aggregate. Eligible compile, grounding, or execution failures can trigger feedback-conditioned replanning. Every repaired proposal is compiled, grounded, executed, and checked again. Clean no-result, ambiguous, and unsupported states terminate without answer-shopping. Training-data collection permits two repair rounds; the controlled runtime evaluation permits one.

4.4 Training from checked executions

The teacher pipeline runs the full runtime on 9,267 training questions. It uses `gpt-5.4-nano` for Step 1 and `grok-4-1-fast-non-reasoning` for Step 2, both at temperature zero; it considers at most two structural plan samples and two feedback repairs. The exact prompt/call contract and its artifact boundary are given in Appendix N. A plan is a positive answerable target only when it passes compile, grounding, execution and release checks, has the required answer shape, and matches the hidden oracle:

$$A(z) = G(z, x, K) \wedge V(z, K) \wedge M(\text{Exec}(z, K), y) \wedge S(\text{Exec}(z, K)). \tag{1}$$

Here G is compile and grounding, V is runtime verification, M is typed oracle match, and S is answer-shape agreement. Runtime verification itself never sees y; the oracle is added only by the offline curator. A plan that executes but has the wrong answer becomes a hard negative. Successful feedback-conditioned repairs and accepted–rejected pairs are stored separately.

The harvest contains 6,860 runtime-verified traces, of which 5,605 answerable traces also match the oracle and answer shape. Seven lack an exportable graph, leaving 5,598 positive plan rows before caps. The remaining artifacts include 1,725 successful repairs, 390 teacher preference pairs, 1,262 verifier-passing hard negatives, and 590 abstention examples. Family and bucket caps produce 2,787 plan-SFT and 1,679 repair-SFT training rows, with 57 and 46 validation rows. The Step-1 understander uses 5,091 training and 96 validation programs. Later self-harvest and preference stages continue from the SFT adapters.

Table 2: Evaluation protocols and the question each is designed to answer.

Protocol	n	Inference path	Purpose
Procurement main	2,025	Understand, conditional graph plan, ground, execute, one replan	End-to-end accuracy after removing the development subset
Development diagnostic	260	Same full runtime	Baselines and paired checkpoint transitions
PACS	922	One algebra-tree call, no repair	Composition and surface robustness
WTQ pristine-unseen	4,344	One guided algebra-tree call with table schema	Cross-domain portability

The student never receives the oracle answer. Direct plan SFT learns $p(z^+ \mid x, u, \mathcal{S})$, where u is the Step-1 briefing and $\mathcal{S}$ is the schema contract. Repair SFT learns $p(z^+ \mid x, u, z^-, f, \mathcal{S})$ from a failed plan and structured feedback. DPO ranks accepted plans above rejected plans for the same input. These objectives explain why an oracle is used during curation without becoming an answer shortcut at inference.

4.5 Single-call algebra planner

PACS and WTQ use a separate planner that emits one recursive expression or an explicit abstention. The algebra has seven types—Records, Values, Groups, Number, Value, Bool, and Ranking—and operators for filtering, projection, grouping, count, sum, extrema, comparison, set operations, and computed-set membership. Validation rejects unknown fields, incompatible children, trees deeper than 16, and trees larger than 64 nodes. The primary PACS protocol uses one temperature-zero call, no repair, and no guided decoding.

Compose-v3 training examples are accepted only when a generated tree type-checks, the production and secondary executors agree, and the result is non-degenerate. Its current export has 12,414 training and 253 validation rows. The WTQ port replaces the procurement record view with a per-table adapter: a typed view is used for predicates and arithmetic, while a raw view preserves the displayed answer strings. Neither protocol invokes the full understander, graph planner, evidence verdict, or feedback loop.

5 Experimental Setup

Table 2 separates the four evaluation protocols. Their percentages are not points on one learning curve: the procurement runs use the full two-stage runtime, whereas PACS and WTQ evaluate the single-call algebra planner.

Procurement main evaluation. The frozen final partition contains 2,285 rows. Its 260-row balanced subset—20 questions from each of 13 buckets—was used for checkpoint selection and pipeline diagnosis. We therefore remove those IDs from the primary column, leaving 2,025 questions: 1,725 answerable and 100 each ambiguous, unsupported, and no-result. We also report the inclusive 2,285-row artifact for continuity. The eight saved arms are closed-book generation, top-40 RAG, plan-guided RAG, the cloud teacher, hybrid Qwen and Llama systems with a cloud understander, and fully local Qwen and Llama systems. All per-item predictions are rescored by the same typed comparator. The stored run directories do not contain a complete command-level manifest, so the inclusive column is a frozen artifact result rather than a claim of reproduction from the current dirty worktree.

Development diagnostic and training ladder. On the 260 balanced questions we compare a deterministic decomposition planner (the KG-only control), RAG, the cloud teacher, and the two local model families. This is a sequential checkpoint comparison, not a matched component ablation: later rungs change both the training pool and objective. For Qwen3-8B and Llama-3.1-8B, the planner ladder is untuned, SFT, self-harvest repair SFT (RSFT), and DPO. The SFT adapters use 4-bit QLoRA on all target modules with rank 64, $\alpha = 128$, dropout 0.05, effective batch 16, learning rate 10^{-4}, and three epochs. RSFT continues

for two epochs at 5×10^{-5}; DPO continues for one epoch with $\beta = 0.1$ and learning rate 5×10^{-6}. Qwen and Llama use the same data recipe. Each learned two-stage arm draws two structural plan samples and allows at most one feedback replan. Since predictions are paired by question, we report exact McNemar tests. There is one training run per checkpoint, so these tests do not measure seed variance. Appendix L states which existing comparisons are interventions, checkpoint ladders, or robustness tests.

PACS. PACS covers seven capability families and three composition levels. The current 922-row manifest has 730 answerable, 138 missing-operator, and 54 empty-result items. Channel A is generated by an independent renderer; channel B is a paraphrase that passes literal and logic gates. Each planner receives the same algebra contract and makes one temperature-zero, prompt-only JSON call without repair. Saved trees are type-checked and executed on the frozen procurement view.

The original channel files treated 36 answerable false or zero-valued rows as no-result. We correct this without another model call by joining saved outputs to the current manifest and re-executing their trees. Thirty-five channel-A and all 36 channel-B predictions are correct under the repaired labels. We use a strict Boolean comparator, so a Boolean never matches integer zero or one. PACS excludes exact test questions and tree hashes from Compose-v3 training, but its near-duplicate gate samples the training trigrams and two rows overlap between Compose-v3 train and validation. We call it a compositional diagnostic rather than a fully sealed OOD benchmark.

WikiTableQuestions. WTQ uses 4,344 human questions over 421 tables absent from its training-table partition (Pasupat & Liang, 2015). The table adapter supplies a dynamic column schema, cell-linking hints, a typed computation view, and a raw projection view. Every arm makes one temperature-zero guided call. We evaluate the Qwen3-8B base, the procurement Compose-v3 checkpoint, an answer-only self-harvest adapter (A), and an adapter trained on translated SQUALL gold programs (C) (Shi et al., 2020). Candidate A trees are kept only when execution matches the answer; C trees are kept only when translation, type checking, execution, and the target agree. The frozen run records 4,709 A examples and approximately 5,423 C examples, although the exact C training snapshot was not preserved in the current Git state. Accuracy is recomputed from saved answer items with the official WTQ evaluator v1.0.2 and CoreNLP-tagged targets, including the tab/newline sanitation used by the frozen runner.

6 Results and Analysis

6.1 End-to-end procurement QA

Table 3 reports the main system comparison. The fully local Qwen pipeline is best on the 2,025 questions outside the development subset, with 1,736 correct predictions (85.73%). It exceeds the cloud teacher by 16.54 points and top-40 RAG by 47.31 points. On paired items, it changes 372 teacher errors to correct answers and loses 37 teacher successes ($p = 9.79 \times 10^{-71}$, exact McNemar). Replacing the cloud understander in the hybrid Qwen pipeline with the local trained understander changes 222 errors to correct and 60 correct items to wrong ($p = 5.01 \times 10^{-23}$). The corresponding Llama comparison also favours the fully local pipeline, indicating that the result is not confined to one model family.

The non-development subset contains 1,725 answerable and 300 non-answerable questions. Fully local Qwen answers 1,436 answerable questions correctly (83.25%) and withholds all 300 non-answerable answers. Closed-book accuracy illustrates why overall accuracy must be decomposed: 360 of its 380 inclusive successes are abstentions and only 20 are correct answers. RAG improves answerable coverage, but a finite retrieved context remains a poor basis for exhaustive aggregation.

6.2 Development checkpoint analysis

The 260-question set supports paired comparisons because every checkpoint uses the same questions and runtime. It does not isolate one training component because the corpus and objective change between stages. Table 4 shows the resulting checkpoint ladder. SFT provides the reliable gain: Qwen improves by 10.77 points and Llama by 23.08. RSFT changes almost equal numbers in each direction. DPO is not significant

Table 3: End-to-end procurement results. The primary column removes all 260 questions used for development and model selection. The inclusive column reproduces the complete frozen artifact. Accuracy includes exact answers and correct non-release.

System	Non-development ($n = 2{,}025$) Correct	Acc. (%)	Inclusive ($n = 2{,}285$) Correct	Acc. (%)
Closed-book LLM	317	15.65	380	16.63
Plan-guided RAG	496	24.49	584	25.56
Top-40 RAG	778	38.42	896	39.21
Cloud teacher runtime	1,401	69.19	1,594	69.76
Hybrid Llama runtime	1,527	75.41	1,736	75.97
Hybrid Qwen runtime	1,574	77.73	1,783	78.03
Fully local Llama runtime	1,681	83.01	1,904	83.33
Fully local Qwen runtime	**1,736**	**85.73**	**1,957**	**85.65**

Table 4: Planner checkpoint analysis on the 260-question development set. Discordant counts and exact McNemar p compare each row with the previous row for the same base.

Base	Checkpoint	Correct / 260	Acc. (%)	Gain/loss; p
Qwen3-8B	Untuned	183	70.38	–
	SFT	211	81.15	31/3; 7.66×10^{-7}
	RSFT	212	81.54	19/18; 1.000
	DPO	217	83.46	20/15; 0.500
Llama-3.1-8B	Untuned	156	60.00	–
	SFT	216	83.08	63/3; 1.30×10^{-15}
	RSFT	215	82.69	8/9; 1.000
	DPO	201	77.31	4/18; 0.0043

for Qwen and is significantly worse for Llama. Later curation stages therefore do not produce a monotonic improvement.

The deterministic KG-only planner obtains 155/260 (59.62%), naive RAG 82/260 (31.54%), strong RAG 75/260 (28.85%), and the cloud teacher 192/260 (73.85%). The best fully local Qwen and Llama runtimes obtain 224/260 (86.15%) and 219/260 (84.23%). Fully local Qwen corrects 73 KG-only errors and loses four KG-only successes ($p = 1.89 \times 10^{-17}$). The KG-only control is strong on count, sum, and extrema but weak on bridge, categorical, factoid, set, and top-k questions. The learned planner supplies coverage that fixed decomposition rules do not.

The capability breakdown in Appendix Table 12 makes this difference more specific. Fixed rules solve 7/20 bridge and 1/20 top-k questions, whereas fully local Qwen solves 14/20 and 18/20. Strong RAG retains factoid performance but solves none of the bridge, extrema, sum, or top-k items. Exhaustive execution helps only after the question has been mapped to the right roles, dependencies, and operation.

6.3 Compositional planning and cross-domain transfer

Table 5 reports the two single-call protocols. Under the corrected PACS manifest, Compose-v3 obtains 78.31% on channel A and 77.11% on channel B. The 1.20-point surface gap is small relative to the improvement over the base and teacher. The result isolates question-to-program generation; it does not include the full runtime's understanding, grounding, evidence, or repair stages.

On WTQ, the procurement Compose-v3 checkpoint raises the base score from 22.51% to 27.33% without WTQ training. Answer-only self-harvest reaches 44.43%, and translated gold-program supervision reaches 51.80%. Official per-item labels give wrong-to-right/right-to-wrong transitions of 387/178 for base to Compose-v3, 833/90 for Compose-v3 to A, and 628/308 for A to C; all three exact McNemar tests have $p < 10^{-18}$. The transfer is therefore not only output-format learning. Procurement training supplies a small planning prior,

Table 5: Single-call typed-algebra results. PACS uses the corrected v1.1 status manifest and deterministic re-execution of saved trees; WTQ uses the official evaluator. Scores across the two panels are not directly comparable.

Evaluation	Planner / supervision	Correct	Accuracy (%)
PACS A	Untuned Qwen3-8B	312 / 922	33.84
PACS A	Cloud teacher	470 / 922	50.98
PACS A	Compose-v3	**722 / 922**	**78.31**
PACS B	Compose-v3	711 / 922	77.11
WTQ	Untuned Qwen3-8B	978 / 4,344	22.51
WTQ	Procurement Compose-v3	1,187 / 4,344	27.33
WTQ	Answer-only self-harvest (A)	1,930 / 4,344	44.43
WTQ	Translated gold programs (C)	**2,250 / 4,344**	**51.80**

denotation-filtered target-domain examples supply a larger gain, and direct program supervision covers questions that self-harvest cannot yet solve.

6.4 What the deterministic checks change

A fixed-plan replay provides a narrow test of hard grounding. Among 136 saved plans with oracles, 89 are correct before grounding and 103 after it. Fourteen plans change from wrong to correct, none changes from correct to wrong, and one becomes executable but remains wrong. All 14 rescues come from inserting the additive-value guard for sums. This shows a direct runtime benefit, not a training effect.

The training-target audit gives the complementary limitation. Of 229 mechanically auditable grounding-related repair targets, 173 retain every mapped question constraint and 56 are flagged; 43 of the omissions concern procurement category. An omitted condition can be redundant on the current snapshot, so the incomplete plan still matches the oracle. The checkpoint gains in Table 4 cannot be attributed specifically to grounding-derived rows because the SFT corpus also changes task language, reader programs, ordinary plans, repairs, and abstention examples.

7 Discussion and Limitations

Answering the research questions. The end-to-end result supports the main system claim: typed learned planning followed by exhaustive KG execution is substantially more accurate than closed-book and retrieval-based access, and the gain remains after removing every question used in the balanced development set. The checkpoint study shows that ordinary SFT supplies most of the stable improvement; RSFT and DPO are not uniformly beneficial across bases. PACS locates a further gain in compositional plan generation, while WTQ shows that the algebra and training recipe can be adapted to a different structured domain. The fixed-plan replay confirms one specific benefit of runtime grounding but does not explain the entire training gain.

Use beyond the benchmark. The architecture is suitable for exploratory audit tasks in which the answer is a deterministic function of a declared data snapshot: complete counts, set relations, extrema, and multi-hop buyer– supplier analysis. Its main practical advantage is division of authority. Language models handle linguistic variation and program proposal; deterministic components control data access, calculation, and release. The same pattern can be reused for other structured public datasets when their identity, value, and provenance conventions are made explicit. WTQ demonstrates portability of the algebra, not portability of the complete procurement runtime.

Evidence and missing-data sensitivity. The implementation records contract identifiers, variable populations, and bounded evidence samples. The main result files, however, retain only predictions and correctness, so the project does not yet report citation precision or recall. Document metadata are extracted,

but no production document inspector verifies source passages. We also did not run a controlled node- or edge-deletion study. Correct abstention on explicit no-result items is evidence for selective behaviour, not a measurement of sensitivity to an incomplete KG. These two requirements from the original project brief remain partial rather than complete.

Data semantics. The graph is only as reliable as its releases and entity resolution. Precision-first matching leaves some organisation aliases separate; projected values are stored names rather than guaranteed legal entities. More than half of evidence edges use an OCID fallback rather than an exact lot link. Currency is stored without conversion, and some additive rows are not GBP. Monetary questions in the benchmark therefore test the declared snapshot semantics and should not be used as real-world financial totals without an explicit currency filter and source review.

Benchmark and artifact integrity. Whole-plan splitting prevents a plan identifier from crossing train and evaluation, but two exact question–answer strings and five deduplication groups cross the graph benchmark boundary. PACS excludes exact test questions and tree hashes from training, but its trigram near-duplicate screen is sampled and Compose-v3 has two train–validation overlaps. The 2,025-row column removes the known 260-question development subset but cannot undo all pipeline decisions informed by earlier diagnostics. The WTQ official run has code hashes and per-item outputs, but the exact C-final training snapshot is no longer present. These facts limit claims of strict sealing and turnkey reproduction.

Statistical and semantic limits. McNemar tests use paired questions and establish that the stored predictions differ systematically; there is only one training run per checkpoint, so they do not estimate seed variance. The cloud and local systems also use different serving stacks. Finally, answer equality on one snapshot does not prove plan equivalence. The constraint audit shows that oracle filtering can admit a plan that omits a redundant condition. One current implementation path makes this risk concrete: the post-compile fidelity net can add a missing year, CPV, or category to the flat query specification, while execution still prefers an already compiled graph plan that does not contain the added atom. Future code should inject the atom into the terminal graph variable or recompile the graph. Future curation should also add counterfactual data worlds or direct constraint-completeness checks.

Responsible use. The records concern public bodies and suppliers, but a technically correct output can still be misleading when notices are incomplete, aliases unresolved, or amounts incomparable. The system is intended for data exploration and audit support, not automated allegations, supplier ranking, or procurement decisions. Released answers should carry the data snapshot, query conventions, population count, provenance sample, and known limitations.

8 Conclusion

This project builds and evaluates an auditable LLM–KG question-answering system for public procurement. It converts versioned OCDS releases into a provenance-preserving property graph, maps questions to typed computations, executes them over complete record populations, and treats abstention as a first-class result. The best fully local pipeline reaches 85.73% on 2,025 non-development procurement questions, while PACS and WTQ show that specialised compositional planning and the typed algebra transfer beyond the main runtime. Deterministic checks improve reliability, but evidence samples, data completeness, entity aliases, currency semantics, and answer-equivalent incomplete plans remain clear boundaries. The main result is therefore not that language models can be made infallible; it is that their role can be restricted to proposing auditable computations whose factual outputs are controlled by explicit data and code.

References

Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. Binding language models in symbolic languages. In *International Conference on Learning Representations (ICLR)*, 2023.

Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Martin Eisenschlos. TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Amita Kamath, Robin Jia, and Percy Liang. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

Kang-il Lee, Segwang Kim, and Kyomin Jung. Weakly supervised semantic parsing with execution-based spurious program filtering. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 6884–6894, 2023. doi: 10.18653/v1/2023.emnlp-main.425.

Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. TAPEX: Table pre-training via learning a neural SQL executor. In *International Conference on Learning Representations (ICLR)*, 2022.

Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, Yifan Zhu, and Anh Tuan Luu. ChatKBQA: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. In *Findings of the Association for Computational Linguistics: ACL*, 2024a.

Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations (ICLR)*, 2024b.

Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida I. Wang, and Xi Victoria Lin. LEVER: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning (ICML)*, 2023.

Open Contracting Partnership. Open contracting data standard. `https://standard.open-contracting.org`, 2024.

Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2015.

Panupong Pasupat and Percy Liang. Inferring logical forms from denotations. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23–32, 2016. doi: 10.18653/v1/P16-1003.

Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.

Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 9895–9901, 2021. doi: 10.18653/v1/2021.emnlp-main.779.

Tianze Shi, Chen Zhao, Jordan Boyd-Graber, Hal Daumé III, and Lillian Lee. On the potential of lexico-logical alignments for semantic parsing to SQL queries. In *Findings of the Association for Computational Linguistics: EMNLP*, 2020.

Ahmet Soylu, Oscar Corcho, Brian Elvesæter, Carlos Badenes-Olmedo, Tom Blount, Francisco Yedro Martínez, Matej Kovacic, Matej Posinkovic, Ian Makgill, Chris Taggart, Elena Simperl, Till C. Lech, and Dumitru Roman. TheyBuyForYou platform and knowledge graph: Expanding horizons in public procurement with open linked data. *Semantic Web*, 13(2):265–291, 2022.

Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 7567–7578, 2020. doi: 10.18653/v1/2020.acl-main.677.

Bailin Wang, Mirella Lapata, and Ivan Titov. Learning from executions for semantic parsing. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2747–2759, 2021. doi: 10.18653/v1/2021.naacl-main.219.

Zilong Wang, Hao Zhang, Chun-Liang Li, Julian Martin Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and Tomas Pfister. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *International Conference on Learning Representations (ICLR)*, 2024.

Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. TaBERT: Pretraining for joint understanding of textual and tabular data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

A Full Procurement Runtime Specification

This appendix makes the main, two-stage procurement system explicit before describing the single-call algebra used by PACS and WTQ. The two representations share a deterministic backend, but they are not interchangeable and are never combined within one reported prediction.

A.1 Intermediate representations

Step 1 returns an *intent program*. Its answer signature declares the operation, value type, and requested answer field; its ordered steps declare literal filters, dependencies, and the final answer step. An unsupported or ambiguous item instead carries a reason and an empty official program. A valid intent is either compiled directly or supplied as a briefing to Step 2.

Step 2 returns the graph-plan interface in Table 6. This representation is used for a multi-hop dependency such as records → emitted suppliers → records bound by those suppliers → a scalar or set answer. The compiler normalises supported planner idioms, constructs a directed acyclic graph, and lowers every executable variable to a backend query specification.

A.2 Online algorithm

For one question, the implemented runtime performs the following steps.

1. Select a schema slice from question keywords. This is rule-based schema selection, not dense retrieval over records.
2. Ask the Step-1 model for an intent program at temperature zero. Validate references, dependency order, return type, answer signature, and any terminal non-answer reason.
3. If the intent is valid, compilable, and not vetoed, compile it deterministically. Otherwise call the Step-2 typed graph planner. In the evaluation protocol, at most two structural samples are considered, and a sample is retried only after a detectable compile or consistency failure.
4. Ground aliases, fields, entity literals, emitted values, downstream bindings, aggregation guards, and the deduplication key. Run schema, conflict, and exhaustive-reduction preflight.

Table 6: Graph-plan intermediate representation used by the full procurement runtime.

Object	Required content	Deterministic checks
Variable	identifier, kind, semantic role, literal constraints, optional emitted field	unique identifier; legal field and value type; no invented root input
Dependency	upstream variable and downstream consumer	reference exists; topological order; no cycle
Relation	source variable/field and target variable/bind field	emitted and bound types agree; no unconstrained bind
Return	input variable, operation, answer field/type, optional grouping or comparator	operation accepts the variable kind; requested count target and direction agree with the question
Guards	exhaustive reduction, deduplication key, additivity and optional population conditions	safe aggregate and declared backend semantics
Abstention	unsupported or ambiguous reason	no answering graph may coexist with a terminal non-answer decision

5. Execute graph variables in dependency order. Derived value sets become hidden membership constraints on downstream record variables. Compute the declared return operation from the resulting record population.

6. Run postflight, answer-sanity, and evidence-release checks. Return an answer card only when the resulting state is releasable. A clean no-result, unsupported, or ambiguous state is final.

7. If a compile, grounding, or execution defect is explicitly repairable, supply the structured failure to the planner and repeat Steps 3–6. Main evaluation permits one feedback replan; teacher collection permits two.

The online path never receives a gold answer. Structural sampling therefore cannot repair an executable-but-semantically-wrong plan unless a runtime check detects its defect.

The current compiler also has a known fidelity gap. Its post-compile check can add a missing year, CPV, or category atom to the flat query specification, but the pipeline executes an existing graph plan in preference to that flat specification. Until the graph is recompiled or its terminal variable is updated, this completion is not a guarantee that the executed graph contains the atom.

A.3 Evidence and release contract

Table 7 distinguishes exact execution state from bounded audit material. This distinction prevents a sample of identifiers from being mistaken for the complete population used by a count or sum.

A.4 Illustrative trace provenance

The candidate running example supplied for Figure 1 is `L2::bridge_join_0502#L2a`: "How many contract notices were published by buyers that have awarded a contract to Dodd Group (Midlands) Limited?" The saved Step-1 intent takes the deterministic compiler branch; Step 2 is bypassed. A no-model replay on the current frozen backend finds 21 source records, emits 16 buyer values, binds those values into 1,940 target records, and returns 1,940, matching the stored oracle. The release includes a limitation: eight of the first 200 matched rows have no recorded supplier field. This is a reconstruction from a saved Step-1 artifact plus current deterministic replay, not a fresh final-test model call. The exact source locations and counts are recorded in `paper/tmlr_v3/figs/src/fig01_runtime_bridge.manifest.json`; a replacement hand-drawn figure should preserve this qualification or use a new rich trace with its own manifest.

Table 7: Information produced and retained by the procurement runtime.

Quantity	Runtime semantics	Retention boundary
Answer and population count	exact deterministic result under the frozen record view	stored in a rich trace; prediction and correctness are retained by the slim main evaluator
Variable trace	grounded constraints, dependencies, emitted field, status, and output size	available when rich tracing is enabled; omitted from the main 2,285-row result files
Contract evidence	matching contract identifiers and rows	exact total count, but identifiers/rows are capped at 200 per execution result
Public evidence view	operation, answer, count, identifiers, and OCIDs	serialises at most 50 identifiers and 50 OCIDs from the runtime bundle
Source documents	URL and document metadata extracted from OCDS	no production document inspector or passage-level citation scorer
Oracle	typed answer used for offline curation and scoring	never included in online prompts or student targets

B Evaluation Protocol Matrix

Table 8 records the decoder and authority boundaries needed to interpret the four result sets. A dash means that the component does not exist in that protocol, rather than that it failed to run.

Table 8: Inference and curation protocols. "Oracle" refers to access during the named procedure, not to whether an oracle later exists for scoring.

Protocol	Plan form	Decoding	Repair	Oracle and saved output
Teacher harvest (9,267)	Step-1 intent; conditional Step-2 graph	temperature 0; two structural samples	at most 2	oracle-hidden online, then oracle/shape gate offline; rich trace and separate training sinks
Procurement main (2,285; primary 2,025)	same full runtime	temperature 0; two structural samples	at most 1	no online oracle; saved evaluator files retain slim per-item predictions
Development (260)	same full runtime; checkpoint varies	temperature 0; two structural samples	at most 1	no online oracle; paired predictions by question
PACS (922)	one recursive algebra tree	temperature 0; primary protocol is prompt-only JSON	none	no online oracle; tree, execution status, answer, and offline score saved
WTQ (4,344)	one recursive algebra tree with per-table schema	temperature 0; JSON-schema guided	none in the reported pristine run	no online oracle; answer items rescored by the official evaluator

The main procurement run directories do not preserve one complete command and checkpoint manifest. The PACS corrected per-item file and the exact WTQ C-final training snapshot are also absent. These are historical artifact gaps, not omitted algorithmic choices; they are stated here to delimit the level of reproduction currently possible.

C Recursive Algebra

The direct-composition language is a closed tree algebra. Programs are limited to depth 16 and 64 nodes. Table 9 gives the public node signatures; predicates include equality, membership, substring containment, existence, numeric bounds, negation, disjunction, and membership in a computed Values set.

Counts and grouped counts deduplicate contract identifiers. Projections remove null and empty values and return sorted distinct strings. Sums use decimal accumulation. Ranking ties are ordered by their stored

Table 9: Recursive-algebra operators. The validator checks compositional types and required arguments before execution.

Node	Signature	Function
`filter`	predicates → Records	filter records
`values`	Records→ Values	project distinct values
`count`, `sum`	Records→ Number	aggregate records
`size`	Values→ Number	set cardinality
`exists`	Records→ Bool	test non-emptiness
`select`, `extreme`	Records→ Value	scalar or extremal lookup
`groupby`	Records→ Groups	grouped count or sum
`argext`, `top`	Groups→ scalar/ranking	rank groups
`combine`	Number^2 → scalar	arithmetic or comparison
`vcompare`	Value^2 → Bool	compare values
`setop`	Values^2 → Values	union, intersection, difference
`gcombine`	Groups^2 → Groups	combine aligned groups
`keys_where`	Groups→ Values	threshold group keys

string keys. A Boolean matches only a Boolean, so `True` is not equal to numeric one. The type checker does not by itself enforce every domain convention: additivity, schema membership, and currency remain separate contracts.

D Data Construction and Schema

Release selection. The loader streams yearly compressed JSON Lines files, skips malformed rows, flattens the required OCDS fields, and retains the latest parseable release per OCID. It also stores the years in which the process was observed. This creates a current snapshot rather than a merged event history.

Entity resolution. Registry identifiers have priority. Platform identifiers are linked only under the same OCID, normalised name, and role, and only if all observations identify one official entity. The second phase uses government lookup, exact name and region, then exact name. Empty normalised names remain singletons. Fuzzy similarity creates review candidates only. Reviewed safe edges are applied by a cluster consistency check. The current artifacts contain 204,711 raw aliases and 131,502 canonical organisations; 82,013 remain singletons.

Values and provenance. The amount source precedence is award, linked contract, then tender. Award- and contract-sourced values may be labelled additive; tender ceilings are not. Numeric zero is retained. Currency is stored but not converted. Evidence nodes retain the source field and source document linkage.

Table 10: Retained graph snapshot. One contract may have several supplier edges.

Node type	Count	Edge type	Count
Contract	215,221	`buyer_of`	215,218
Organisation	131,502	`supplier_of`	334,063
CPV	3,870	`categorized_by`	164,691
Evidence	535,731	`evidence_for`	1,326,240

E Benchmark Construction and Integrity

The initial procurement generator creates executable answer specifications from the graph. Gate A checks conflicts, identifier integrity, deterministic execution, and aggregate sanity before surface generation. A second language-model stage writes the question; Gate B checks that visible filters and operations agree with the

specification. The initial artifacts contain 10,000 answer specifications, 9,053 generated questions, and 9 Gate-B failures, yielding 9,044 rows. Later curation adds answerability twins, whole-plan split assignment, class balancing, and surface variation to obtain 12,828 graph-benchmark rows: 9,267 train, 671 development-select, 556 development-tune, 49 smoke, and 2,285 final-test rows.

Plan identifiers do not cross from train to final test in the stored split. A separate string audit nevertheless finds two identical question–answer pairs and five deduplication groups across that boundary. The split should therefore be described as plan-ID separated, not fully duplicate-free.

Table 11: Composition of the 2,285-row frozen procurement partition. The 260-row development subset contains 20 rows from each bucket and is removed by identifier to form the 2,025-row primary column.

Bucket	Rows	Bucket	Rows	Bucket	Rows
Ambiguous	120	Boolean	121	Bridge join	350
No result	120	Categorical	120	Comparison	306
Unsupported	120	Count	250	Factoid	250
Min/max	107	Set	93	Sum	200
Top-k	128	Answerable total	1,925	Grand total	2,285

Table 12: Capability breakdown on the 260-question development diagnostic (20 items per row). Exact counts are reported because the balanced cells are small.

Capability	KG-only	RAG-40	Local Q	Capability	KG-only	RAG-40	Local Q
Ambiguous	15	5	19	Comparison	8	3	17
No result	20	13	20	Count	17	3	20
Unsupported	20	20	19	Factoid	5	17	15
Boolean	11	0	14	Min/max	18	0	20
Bridge join	7	0	14	Set	9	1	15
Categorical	5	13	13	Sum	19	0	20
				Top-k	1	0	18

F PACS Details

PACS instantiates seven families at levels L1–L3. Construction samples executable typed trees, runs the production and separate evaluators, requires answer agreement and non-degeneracy, and excludes exact tree hashes found in Compose-v3 training or validation. Missing-operator and no-result controls are added separately. The evaluated surface files contain 922 rows. Table 13 reports the actual status and exposure labels used by evaluation.

Table 13: Corrected PACS v1.1 composition. "Seen" refers to the declared program-shape label, not exact tree overlap.

Status	Rows	Shape label	Rows
Answerable	730	Seen	431
Unsupported	138	Unseen	491
No result	54	Total	922

The original surface channels treated 36 answerable false or zero-valued denotations as no-result. The corrected v1.1 manifest restores them to the answerable class. Saved predictions were joined by item identifier and their trees were re-executed; no model was called. This changes Compose-v3 channel A from 687 to 722 correct and channel B from 675 to 711 correct. The repository does not retain the original status-transformation script, so the paper artifact records this deterministic correction explicitly. The seal implementation also differs from its comments. It checks exact question and tree hashes, entity anchors, and a sampled trigram

screen, but it does not implement a separate template-identity gate and compares only every seventh training trigram set. These facts motivate the qualified description in Section 5.

The declared seen/unseen labels mix answerable and status rows, so they are a difficulty diagnostic rather than a pure measure of logical-form novelty. Exact question and tree-hash overlap with the PACS test is absent; this is a narrower claim than saying that every program shape is unseen.

G WikiTableQuestions Evaluation

The WTQ evaluation uses 4,344 questions over 421 test tables absent from the training-table partition. Each table is exposed through two aligned views. The typed view adds deterministic numeric, date, year, month, and normalised-string columns for computation; the raw view preserves the strings returned to the official scorer. Row identifiers and derived columns are never shown as answer values.

For official scoring, each predicted answer item first replaces tab and newline characters with a space. The question identifier and cleaned items are then joined with tab delimiters and passed to `scripts/wtq/official_evaluator.py` version 1.0.2 with the original CoreNLP-tagged targets. This detail changes three borderline items relative to calling the normaliser directly. Table 14 records the official counts and paired transitions reconstructed from saved answer items.

Table 14: WTQ pristine-unseen official evaluation. Gain/loss compares each row with the row immediately above using exact McNemar tests.

Planner or supervision	Correct / 4,344	Accuracy (%)	Gain/loss; p
Untuned Qwen3-8B	978	22.51	–
Procurement Compose-v3	1,187	27.33	387/178; 8.50×10^{-19}
Answer-only self-harvest (A)	1,930	44.43	833/90; 1.77×10^{-151}
Translated gold programs (C)	2,250	51.80	628/308; 6.20×10^{-26}

The A run records 4,709 answer-matched examples. The C run used approximately 5,423 translated program examples selected by translation, type, execution, and answer agreement. Its exact training snapshot is not present in the current repository: the historical commit, current file, and staged file contain different later reconstructions. The official per-item predictions remain available, but this missing snapshot prevents a turnkey retraining claim.

H Grounding and Target Audits

The fixed-plan replay executes the same saved proposal before and after deterministic grounding on the same frozen backend. Of 136 plans with typed oracles, 89 are correct before grounding and 103 after it. Fourteen move from wrong to correct, none moves from correct to wrong, and one previously non-executable plan becomes executable but remains incorrect. All 14 rescues are sums for which grounding inserts the additive-value guard. This isolates one runtime transformation; it does not estimate the effect of grounding-derived rows on a trained model.

The teacher artifacts contain 237 successful repairs attributed to schema, runtime-specification, or entity-grounding failures. Of 229 targets that can be checked mechanically against mapped question constraints, 173 contain every mapped constraint and 56 are flagged. Category is absent from 43 flagged mappings, followed by buyer (6), supplier (3), signed-date guard (3), title (2), and CPV (2); categories may overlap within one target. A missing condition can be extensionally redundant on one graph snapshot, allowing an incomplete plan to match the answer oracle. This is why the paper does not equate denotation agreement with full semantic equivalence.

I Training Data and Configurations

Table 15 separates candidate harvest counts from final exported training splits. A candidate count is not an accuracy and overlapping objectives do not create new unique questions.

Table 15: Graph-plan teacher harvest and capped exports.

Artifact	Harvest rows	Exported train/validation
Runtime-verified traces	6,860	–
Runtime-verified and oracle-correct	5,605	–
Direct graph targets	5,598	2,787 / 57
Successful repair targets	1,725	1,679 / 46
Preference pairs	390	390 / –
Abstention targets	590	258 included in direct train
Step-1 intent targets	–	5,091 / 96

Table 16 records the configurations actually consumed by the reported checkpoints. Every row uses 4-bit QLoRA on all target modules, rank 64, $\alpha = 128$, dropout 0.05, bfloat16 compute, effective batch 16, cosine decay, and one CUDA process. Step-2 SFT trains on 4,466 rows: 2,787 direct and 1,679 repair targets. Its configured validation set contains the 57 direct-plan rows; the separately exported 46 repair validation rows are not selected by the training YAML.

Table 16: Training and continuation configurations. Q/L denotes matched Qwen3-8B and Llama-3.1-8B variants.

Stage	Initialisation and train rows	Objective	Epochs	LR	Cutoff / validation
Step-2 SFT (Q/L)	base; 2,787 plan + 1,679 repair	token SFT	3	10^{-4}	6,144 / 57 plan
Step-1 SFT (Q/L)	base; 5,091 intent programs	token SFT	3	10^{-4}	6,144 / 96
Qwen RSFT	Qwen SFT; 3,019 plan + 3,169 repair	token SFT	2	5×10^{-5}	6,144 / 65 plan
Llama RSFT	Llama SFT; 3,026 plan + 3,190 repair	token SFT	2	5×10^{-5}	6,144 / 66 plan
Qwen DPO	Qwen RSFT; 390 teacher + 689 on-policy pairs	sigmoid DPO, $\beta = .1$	1	5×10^{-6}	6,144 / none
Llama DPO	Llama RSFT; 390 teacher + 693 on-policy pairs	sigmoid DPO, $\beta = .1$	1	5×10^{-6}	6,144 / none
Compose-v3	fresh Qwen; 12,414 trees	token SFT	2	10^{-4}	4,096 / 253
WTQ A	fresh Qwen; 4,615 answer-matched trees	token SFT	2	10^{-4}	4,096 / 94
WTQ C	fresh Qwen; about 5,423 translated programs	token SFT	2	10^{-4}	4,096 / snapshot missing

Training logs record Transformers 5.6.0 and the expected example and optimiser-step counts. The evaluation server logs record vLLM 0.24.0, seed 0, bfloat16, maximum sequence length 8,192, LoRA rank 64, and disabled Qwen thinking. They do not record a formal GPU-model manifest. The paper therefore reports one-device execution without claiming a hardware model that cannot be recovered from the preserved logs.

J Artifact and Reproduction Notes

The implementation entry points are:

- `pipelines/01_ingest.py` through `pipelines/04_extract.py`, followed by `pipelines/40_build_kg.py` and `pipelines/41_validate_kg.py`;
- `scripts/apply_safe_er_merges.py` for the reviewed cluster-safe ER layer;
- `scripts/run_teacher.py` and `scripts/export_llamafactory.py` for graph-plan curation and export;
- `scripts/run_compare.py` for the full runtime, and `scripts/run_compose_probe_eval.py` for direct composition;
- `scripts/analyze_grounding_impact.py` for the fixed-plan replay.

Paper-side derived counts and provenance notes are stored in `paper/tmlr_v3/artifacts/derived_metrics.json` and `paper/tmlr_v3/artifacts/ARTIFACT_LEDGER.md`. The compact KG-only development summary records the SHA256 values of the 260-row input, runner, full per-item result, and source summary without copying the 9 MB diagnostic result into the paper directory.

The prompt/call contract, source and configuration hashes, selection authority, serving settings, and ablation classification are recorded in `paper/tmlr_v3/artifacts/REPRODUCIBILITY_MANIFEST.md`. The historical teacher summary stores the Step-1 and Step-2 model names but omits the command, prompt hash, worker count, and most call parameters. Those fields are explicitly marked as reconstructed from the audited runner. The main procurement run directories likewise lack a unified arm-to-checkpoint manifest, and the currently linked adapter directories are not portable repository artifacts.

Full production builds use canonical output paths. Partial year, limit, or sample runs require an explicit non-canonical output location, preventing diagnostic runs from replacing the canonical tables. The current working tree contains staged data artifacts and unstaged correctness fixes; a release version must record a source commit, data hashes, checkpoint hashes, decoder settings, and scorer version together. We bind the present paper to its enumerated artifacts rather than claiming that every historical summary can be regenerated from an unspecified working tree.

K Project-Brief Traceability

Table 17 maps the original project requirements to implemented evidence and remaining gaps. It distinguishes a completed component from one that is designed or only partly evaluated.

L Ablation Coverage and Causal Scope

Table 18 separates a controlled intervention from a checkpoint or robustness comparison. This distinction matters because a fixed test set alone does not make a training ladder a component ablation.

The existing artifacts therefore support two restrained conclusions. First, deterministic grounding can repair a specific class of saved sum plans at runtime. Second, full SFT improves the paired end-to-end checkpoint on both base models. They do not identify the fraction of the SFT gain caused by grounding-derived rows. A causal claim would require corpora matched for unique question, family, depth, output type, target length, and optimiser tokens while adding only ordinary repair, grounding-derived repair, or grounding-derived preferences. Other missing component tests include guided versus unconstrained decoding, online grounding on/off, release checks and feedback repair on/off, and deterministic fast path versus forced Step 2. We list these as needed experiments rather than retroactively labelling existing comparisons as ablations.

M Recommended Additional Evaluation

The matched grounding-data ablation should use at least three seeds and paired decoding on one frozen test. Before training, all four corpora should be matched by unique question, family, program depth, output type, target length, and total optimiser tokens. After training, each output should be labelled for field, role,

Table 17: Coverage of the original LLM–KG procurement project brief.

Requirement	Status	Evidence and boundary
Structured/semi-structured ETL and KG schema	Implemented	versioned OCDS ingestion, rich extraction, four node types, four edge types, validation before write
Entity resolution and aliases	Implemented	registry-first and precision-first deterministic rules, reviewed safe merges, alias audit, unresolved singletons
Natural language to executable KG computation	Implemented	typed Step-1 intent, deterministic fast path, conditional graph planner, grounding and executor
Multi-hop, aggregation, temporal and set reasoning	Implemented	bridge dependencies, count/sum/extrema/top-k/set/Boolean operations, year and date constraints
Evidence-linked reasoning trace	Partial	variable traces and contract identifiers exist; main evaluation stores slim outputs and no source-passage citation score
Reliability controls	Implemented	typed/schema checks, exhaustive execution, pre/postflight, answer sanity, explicit abstention, bounded replan
Factual, compositional and non-answer evaluation	Implemented	procurement final partition, PACS, and explicit ambiguous/unsupported/no-result classes
Citation accuracy	Not evaluated	document metadata exist, but there is no production passage inspector or citation gold set
Sensitivity to KG incompleteness	Not evaluated	no controlled node/edge deletion or missing-field intervention study
Closed-book, RAG, LLM+KG, and deterministic controls	Implemented	all four families are reported; the deterministic control is rule/decomposition over the KG rather than SPARQL/Cypher

Table 18: What each existing comparison holds fixed and what it can establish.

Evidence	Held fixed	Changed	Interpretation
Pre/post grounding replay (136)	saved question, plan, KG, executor, oracle	deterministic grounding transformation	narrow runtime intervention; chiefly the additive-sum guard
Checkpoint ladder (260)	questions, scorer, runtime budget, base within family	corpus, initial adapter, and objective across stages	sequential comparison, not matched ablation
Hybrid/fully local runtime	questions and downstream runtime	Step-1 model and historical deployment configuration	system-configuration comparison; manifest limits causal attribution
PACS A/B	program, oracle, and planner	surface rendering	paraphrase robustness
WTQ base/v3/A/C	test set and official scorer	source and amount of supervision	supervision ladder, not single-component ablation

operator, return type, dependency, explicit-constraint recall, hidden guard, validity, executability, answer correctness, and abstention. Reporting a transition matrix for each decision reveals whether an apparent accuracy gain came from the intended hard signal or from generic JSON and domain-language adaptation. Predicate-sensitive counterfactual worlds should be part of the test so that dropping any evaluated constraint changes the denotation.

N Reader, Planner, and Repair Interfaces

The prompt builders and provider schemas are generated by `src/procurement_graph/reasoning/typed_planning.py`. Its audited SHA256 is recorded in `paper/tmlr_v3/artifacts/REPRODUCIBILITY_MANIFEST.md`, together with the runner and configuration hashes. The historical teacher summary preserves both model names but not a full command or prompt version. Table 19 therefore distinguishes stored facts from settings

reconstructed from the audited runner. The code is the canonical source for exact whitespace, dynamic schema context, and enum expansion.

Table 19: Teacher prompt and call contract. "Stored" is present in the harvest summary; "reconstructed" is implied by the audited runner and model-dependent prompt selection.

Call	Model	Output control	Sampling and visibility
Step-1 intent	`gpt-5.4-nano` (stored)	strict intent-program JSON Schema	temperature 0; question and schema slice; no records or oracle
Step-2 graph	`grok-4-1-fast-non-reasoning` (stored)	reconstructed `lean/optional` strict graph schema	temperature 0; at most two structural samples; question, Step-1 output, and schema slice
Repair reading	Step-1 model	fixed eight-section text scaffold	called for semantic repair; hidden-answer fields removed
Repair graph	Step-2 model	strict repair/graph JSON Schema	at most two teacher-harvest repairs; failed graph and structured feedback, but no oracle value

The shared API client uses a 60-second timeout and allows three retries after the initial request, with waits of 2, 4, and 6 seconds. Rate limits remain retryable; most other 4xx failures terminate immediately. The runner does not set an explicit output-token limit. These call settings are reconstructed from the current client because they were not copied into the historical summary.

N.1 Step-1 understander

The exact system instruction is:

> "You are the typed intent programmer for a UK public-procurement KGQA system. Convert the question into a small computation program. Do not answer the question. Use only information explicitly present in the question. The output shape is enforced by a strict JSON schema."

The user JSON contains the keys `question`, `task`, `allowed_slots`, `operation_guidance`, `hard_rules`, and `retrieved_schema_context`. The required output contains:

```
answer_signature: operation, value_type, answer_field_text
program: ordered typed steps with literal filters and dependencies
answer_step: id of the final step
unsupported_or_ambiguous: explicit reason or empty
```

Allowed literal slots are publication year/range, CPV code and label, procurement category limited to goods/services/works, buyer, supplier, exact quoted title, date, and numeric amount. Root steps cannot invent an "all records" input. Later steps may reference only earlier identifiers. Bridge intents must construct source records, a distinct entity set, a dependent target set, and a final operation. Future years remain executable no-result queries rather than predicted abstentions. If the question is unsupported or ambiguous, the official program is empty.

N.2 Step-2 graph planner

Step 2 is called only when the Step-1 program cannot take the deterministic fast path or is vetoed. The teacher model name selects the lean prompt rather than the larger capability-card prompt. Its exact system instruction is:

> "You are the structured graph planner for a UK public-procurement KGQA system. Turn the question and the Step-1 briefing into an executable `graph_plan` (variables + return). A strict schema fixes the output shape; fill the fields faithfully and never answer the question."

It receives the original question, the Step-1 briefing, schema context, and instruction list. Its JSON graph contains a flat variable array, relations, a return record, a template label, and an optional abstention reason. Record filters contain only literal constraints from the question. Derived phrases are represented as variables and relations, not copied into filter slots. A bridge must emit a typed entity set and bind it into the target record variable. Money sums must request the additive-value guard. Unused schema fields take declared empty values rather than invented semantics.

The planner never receives records, an execution result, or the oracle answer. The consistency checker compares the graph with explicit years, CPV codes, categories, titles, numeric literals, buyer/supplier roles, and the requested count target. A structural resample appends only the diagnostic that compilation or consistency failed and asks for a different, simpler decomposition.

N.3 Feedback-conditioned repair

An eligible runtime failure produces a structured record containing its stage, code, path, and message. Repair reading is instructed to revise the understanding from the failed graph and feedback without answering or inferring a reference answer. Graph repair is instructed to diagnose the failed graph, choose an allowed action, and emit a graph under the original schema. The feedback sanitizer removes gold/reference/expected/oracle-answer keys. Repair SFT conditions on the original question, Step-1 briefing, failed graph, and this visible feedback. It targets the accepted repaired graph. Oracle mismatch feedback states only that external validation failed and omits the expected value. Clean no-result and unsupported cases terminate; they are not repeatedly replanned in search of a non-empty answer. The teacher collection budget is two repairs, while the 260-question evaluation budget is one.

N.4 Training examples and information boundaries

Direct-plan SFT serialises $(x, u) \mapsto z^+$. Repair SFT serialises $(x, u, z^-, f) \mapsto z^+$. Preference training uses the same question and briefing with accepted and rejected graph completions. Neither the oracle answer nor the candidate's returned value is included in the student input or target. This boundary is important for interpreting the learned signal: the model can imitate the selected planning decisions and the response to visible failure feedback, but it is never directly told which individual decision made the denotation correct.

Offline selection nevertheless uses more than runtime grounding. A direct answerable target must pass execution/release checks, match the hidden answer oracle, and have the expected output shape. A runtime-passing but answer-wrong graph enters the hard-negative path. Its repair becomes a repair-SFT or DPO positive only after deterministic re-execution matches the same hidden gates. Thus grounding defines an executable boundary, whereas the oracle filters denotational correctness; neither proves that every question constraint is present in the plan.

The teacher writer is append-only and resume is not transactional. It records the trace before the downstream SFT/DPO sinks, while resume skips every identifier already in the trace file. A failure between those writes can therefore leave an incomplete downstream corpus. The emitted metadata string `acceptance: executor_verifier` is also stale: the actual control flow has already applied the oracle/shape branch before direct SFT is written. Reproduction should use the runner logic and per-sink counts rather than that label.